

中国机器人及人工智能大赛四足仿生机器人中型组比赛
技术报告

学 校： 延安大学西安创新学院

院 系： 数据科学与工程学院

团队名称： Salvator 队

指导老师： 闫文耀 吴晓晖

队 员： 孟德森 杨孜萌 霍楠

日 期： 2026 年 6 月 2 日

附件 3

第二十八届中国机器人及人工智能大赛参赛选手告知书

为保障第二十八届中国机器人及人工智能大赛（以下简称“大赛”）顺利举办，规范参赛流程，明确参赛选手责任与义务，切实维护赛事公平、公正、安全、有序的运行环境，现就相关事项告知全体参赛选手，请认真阅读并严格遵照执行：

一、报名流程要求

参赛选手须依照规定途径访问大赛官方报名系统，如实、完整地填报并提交相关报名材料，确保所有材料真实、有效且符合赛事规定。进入报名系统进行报名，材料提交后，须进行最终核对，确认无误后方可提交。

网址：<https://www.cairobot.com>

二、报名信息核对要求

所有参赛信息（包括个人信息、团队信息、作品信息等），均须由参赛选手在提交前仔细核对、反复校验，确保准确无误。报名系统关闭后，不再受理任何形式的信息修改。如因个人操作失误导致信息错误，应及时联系所属省赛组委会联系人申请退回修正。各省赛联系人、赛区工作群、预计省赛时间等具体信息详见附件 1。

三、身体健康状况申报要求

参赛选手须如实申报本人身体健康状况，审慎评估自身身体状况，确认能够适应本次大赛的竞赛强度及相关活动要求，不存在不适宜参加集体外出、长时间备赛、现场竞赛及相关实践活动的疾病（如心脏病、高血压、传染性疾病等）或身体不适。如具有特殊体质、既往病史、过敏史等情况，须提前主动向所在派出单位报备，不得隐瞒或虚报。派出单位应结合选手身体状况进行综合评估，确定其是否适宜参赛，以保障选手在赛事期间的人身安全与健康。

四、赛事纪律要求

大赛严禁一切违规违纪行为，具体规定如下：

严禁一稿多投，同一作品不得同时投递其他赛事，一经发现，将取消其参赛资格及已获得的全部成绩与荣誉；

严格规范学术行为，杜绝任何学术不端行为（包括但不限于抄袭、剽窃他人成果、篡改实验数据等），一经查实，将取消参赛资格、撤销所获奖项，并保留通报权利；

严禁任何污蔑、诋毁参赛学生、裁判及赛事工作人员的行为，一经发现，立即终止其参赛资格，情节严重的，将移交有关部门处理，并保留追究其相应责任的权利；

所有违规违纪行为的处理结果均将通过官方渠道以书面形式公示，确保处理过程与结果的公平、公正、公开。

参赛选手完成报名即视为已阅读、理解并同意遵守本告知书全部内容。如因违反本告知书相关要求而产生任何后果，均由参赛选手本人及所在派出单位自行承担。

特此告知。

团队编号：

参赛选手签字：霍楠 杨政翔 孟德森

第二十八届中国机器人及人工智能大赛组委会

2026 年 5 月 19 日

摘要

本项目以 2026 中国机器人大赛暨 RoboCup 机器人世界杯中国赛—四足仿生机器人快递运送赛（中型组）为应用载体，围绕智能物流末端配送的实际需求，开展高动态、高稳定性、高自主化四足仿生机器人的整体设计、系统研发与工程实现。机器人主体采用高强度铝合金轻量化框架，配备 12 自由度准直驱行星减速驱动单元，搭载高性能直流无刷伺服电机，在动力输出、运动精度与结构刚性上实现综合优化，具备优异的地形适应能力、越障能力与动态运动性能。

为保障复杂路况下的作业可靠性，系统融合 IMU 惯性姿态感知、实时姿态估计与抗外力扰动动态平衡控制算法，可在坡道、台阶、崎岖路面等场景下维持稳定姿态；同时创新性设计带坡度约束的快递承载机构，有效抑制运输过程中货物滑移、倾倒与脱落，显著提升任务安全性与完成度。

控制系统基于 Linux 嵌入式平台，构建以 Qt、LCM 轻量级通信、OpenCV 视觉处理为核心的一体化软件架构，实现环境感知、路径规划、视觉循迹、运动控制与任务调度的有机协同。软件层面采用模块化分层架构与有限状态机（FSM）设计，完成自主站立、动态平衡、轨迹跟踪、越障决策、快递精准投放、自动返回与定点停靠等全流程自主作业，具备逻辑清晰、切换可靠、鲁棒性强等特点。

该机器人整机尺寸约 485mm×275mm×300mm，重量约 10.5kg，能够在无人工遥控条件下完成赛事全部规定任务。本研究不仅充分满足机器人大赛的技术指标与竞技要求，更为智能物流、应急配送、复杂环境作业、室外自主巡检等领域提供了一套可行、高效、高可靠性的四足仿生机器人技术方案，具有较高的理论研究意义与工程应用价值。

目录

摘要..... 0

一、指导老师介绍 1

二、队员介绍..... 1

三、机器人详细设计文档..... 2

1.设计理念 2

2.控制系统框架 2

四、对比赛规则的认识 3

五、机器人硬件构成..... 4

1.驱动与执行..... 4

3. 控制层 4

4. 通信层 4

六、机器人尺寸及重量 5

七、外观机械图 5

八、附录..... 错误!未定义书签。

一、指导老师介绍

1.姓名：闫文耀

•性别：女

•职务：数据科学与工程学院院长

•办公地点：长安校区 1202 室

•基本信息：吉林市人，九三学社社员，博士研究生（在读），教授，数据科学与工程学院院长、陕西省计算机教育学会常务理事、陕西省科技特派员（自然人科技特派员）、西安市长安区第十九届人大代表（2022 年—2026 年）。

•主要研究方向：智能信息系统、数据挖掘

•主讲课程：《数据库原理与应用》《C 语言程序设计》《操作系统》等。

•科研项目：陕西省 2022 年科技计划项目“基于智能推荐的农副产品一体化电子商务平台系统及应用”（2022FP-40），主持

西安市科技局 2023 年重点产业链技术攻关项目“智慧城市多模态场景感知关键技术研究及应用”（23ZDCYJSGG0023-2023），主持

校级 2023 年科研项目“基于深度学习的情绪识别智能分析系统”（2023KYZX01），主持

校级 2024 年科研专项“西安理工大学-延安大学西安创新学院脑机接口与智慧医疗联合研究中心”，主持

2. 姓名：吴晓晖

•性别：男

•基本信息：生于 1994 年 12 月，山西省永济人，中共党员。2019 年 7 月毕业于内蒙古科技大学，获得计算机技术硕士学位。现为数据科学与工程学院专职教师。。入职以来，获得优秀毕业设计指导教师、优秀实习指导教师等荣誉称号，近两年指导学生参加《中国大学生计算机设计大赛》获得国家三等奖以及省二等奖、三等奖。参加校级青年教师讲课比赛获得三等奖。入职以来，先后发表科研论文 3 篇，参与省、市教育厅项目多项。

•主要研究方向：目前主要从事大数据、区块链访问控制等方面的研究。

•主讲课程：C/C++程序设计、高级语言程序设计、面向对象程序设计、算法分析与设计等课程。

二、队员介绍

1.姓名：霍楠

专业：数据科学与工程学院 - 计算机科学与技术

分工：负责 Linux 系统部署与环境搭建、Qt 应用开发、视觉循迹与目标识别算法实现、仿生步态规划与运动控制、任务状态机架构设计与编程实现，完成系统封装部署及赛事现场调试、故障诊断与性能优化，保障机器人全流程自主稳定运行。

2.姓名：杨孜萌

专业：数据科学与工程学院 - 计算机科学与技术

分工：负责机器人全系统集成与联合调试，统筹机械结构、电控硬件、软件算法多模块协同优化；主导现场部署、实时故障诊断与任务闭环测试，保障软硬件接口稳定、步态精准、任务流程可靠；全程支撑系统综合验证与赛事现场技术保障，确保机器人稳定高效完成自主快递运送全任务。

3.姓名：孟德森

专业：数据科学与工程学院 - 人工智能

分工：主导机器人整机机械架构与腿部运动学优化，创新防脱落夹持机构以提升动态稳定性。统筹电气系统集成，完成驱动、传感及视觉模块的精密布线与 EMC 处理；搭建硬件通信链路并主

导软硬件联合调试，保障系统高可靠运行。

三、机器人详细设计文档

1.设计理念

本机器人是一款轻量型、高动态性能的四足机器人。装备了超高性能直流无刷电机，使机器人具有更强劲的动力。同时，紧凑的关节驱动器结构及其腿足设计，使其运动更高效、更灵活，另外，本机器人内部高性能的算法可以进行抗外力扰动平衡控制，具备良好的地形适应能力。机器人上方的快递盒投放快递的一侧有一定坡度，防止机器人在上台阶或崎岖路段因颠簸导致快递快递滑落。

本产品针对快递配送进行优化，使四足机器人更加适用于快递的配送，可减少物流的配送成本，提高配送效率。

2.控制系统框架

第一步要安装 Ubuntu 系统，若使用虚拟机需要先对虚拟机进行配置

- (1) 在安装 Ubuntu 系统时需要将用户名配置成 user，密码设置为 123456
- (2) 配置成桥接模式，将 usb 控制器设置为兼容 3.0

第二步安装 QT

注意在 ubuntu 系统打开时，系统默认右侧键盘锁打开，数字无法输入，需要点击一下数字锁。

查看是否正确登录

- (1) 将 qt 的库添加

在 home 目录下存在一个隐藏文件.bashrc

在 home 目录下使用 `sudo gedit .bashrc`

- (2) 在文件最后添加

```
export LD_LIBRARY_PATH=/home/user/Qt/5.10.0/gcc_64/lib:$LD_LIBRARY_PATH
```

然后执行下 `source ~/.bashrc`

- (3) 添加 lcm 库，opencv 库

添加 lcm 依赖库（在联网环境下进行）。lcm 是一种轻量级通信，可以在软件间进行通信和在 IP 地址之间进行信息传输。

下载 lcm1.3.1 包进行解压（文件夹附带），然后进入解压的目录下，右键打开终端。

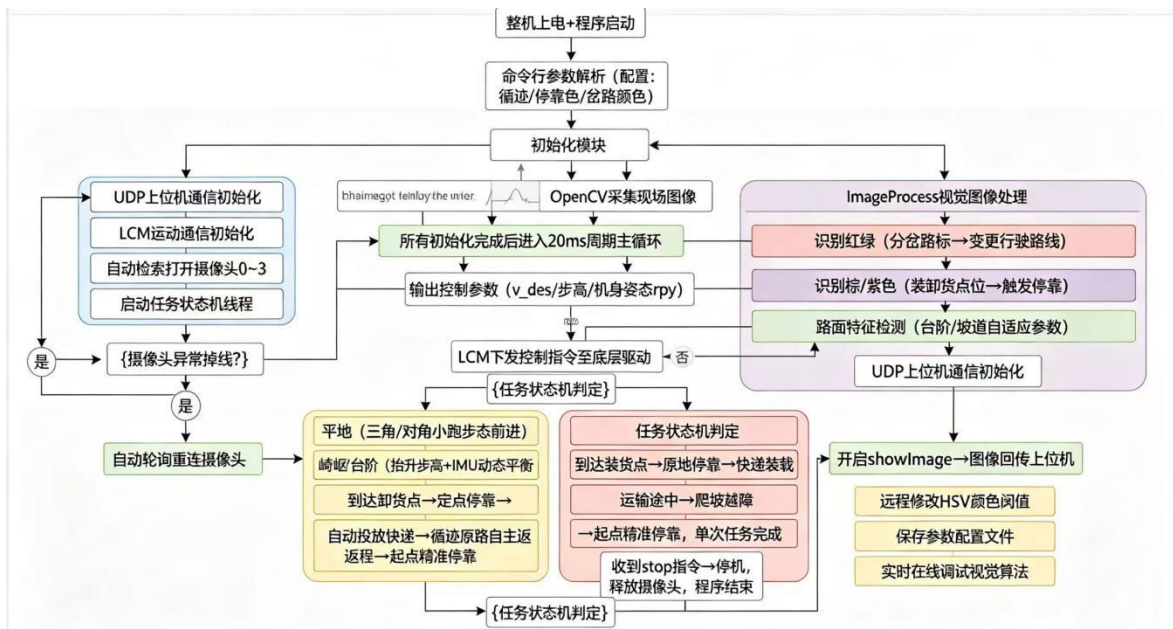
测试 lcm 通信、配置 opencv 库

第三步封装软件，进行如下操作。

- (1) 将封包软件（文件夹内包含）复制到需要封装的软件对应的文件夹

- (2) 在 home/user 目录下使用 Ctrl+H 找到隐藏文件 .bashrc
- (3) 执行如下命令打开.bashrc 文件，在文件最底部输入输入 qt 的 gcc 路径，保存运行
- (4) 对软件进行封包(实际就是运行 linuxdeployqt 软件 将需要封包的软件名作为参数传递进去)使用封包指令 ./linuxdeployqt track -appimage (中间的参数 track 就是被封装的软件)
- (5) 执行程序

四、四足仿生机器人快递任务执行流程图



五、对比赛规则的认识

本赛项为四足仿生机器人快递运送赛（中型组），核心任务：机器人从起点出发，沿规定路线自主行走，完成快递装载、平稳运输、精准投放、越障/上台阶、返回停靠全流程，在限定时间内完成任务并保证快递不滑落、姿态稳定、停靠精准。

六、四足仿生机器人基本技术与机械构成

1. 基本技术

- 仿生步态：三角步态、对角小跑，适应平地/坡道/台阶。
- 动态平衡：基于 IMU 姿态反馈，实时补偿重心，抗扰动。
- 运动控制：FOC 矢量控制驱动无刷电机，力矩/位置双闭环。
- 环境感知：视觉识别路径/投放区/障碍；惯导提供姿态与里程计。
- 软件架构：Linux+Qt+LCM+OpenCV，模块化、低延迟通信。

2. 机械构成

- 躯干：铝合金主体，承载电源、主控、驱动板、快递盒。
- 腿部：4 条腿 $\times$ 3 自由度=12 自由度，含髋俯仰/髋横摆/膝俯仰。
- 驱动：准直驱行星减速结构，紧凑高刚性。
- 轴承：深沟球轴承，保证转动顺畅、寿命稳定。
- 快递机构：带坡度固定仓，防颠簸掉落，便于自主投放。
- 尺寸重量：约 $485 \times 275 \times 300\text{mm}$ ，约 10.5kg，轻量化高刚性。

七、机器人硬件构成

1. 驱动与执行

- 无刷直流电机：高功率密度、大扭矩、响应快。
- 行星减速驱动器：紧凑集成，提高输出扭矩与控制精度。
- 12 路关节驱动：每条腿独立伺服，实现复杂步态。

2. 感知层

- IMU 惯性测量单元：提供姿态角、角速度、加速度，用于平衡。
- 视觉摄像头：OpenCV 处理，识别路径、投放区、障碍。
- 电机编码器：实时关节位置/速度反馈，闭环控制。

3. 控制层

- 上位主控：Linux 环境，运行 Qt 界面、LCM 通信、视觉算法、任务调度。
- 运动控制器：接收指令，下发 FOC 电流环、位置环、步态生成。
- 电源管理板：稳压、保护、状态监测，保证长时间稳定。

4. 通信层

- LCM 轻量级通信：进程间/多板间低延迟数据交互。

- USB 3.0 兼容：高速传输图像与调试指令。

八、机器狗尺寸及重量

表1 机器人基本规格表

材质	铝合金
尺寸	约 485×275×300mm
重量	约 10.5 kg
自由度	12
本体结构	准直驱行星减速结构
轴承	普通深沟球轴承
平台	Linux

九、机械原理图



图1 机器狗站立模样图

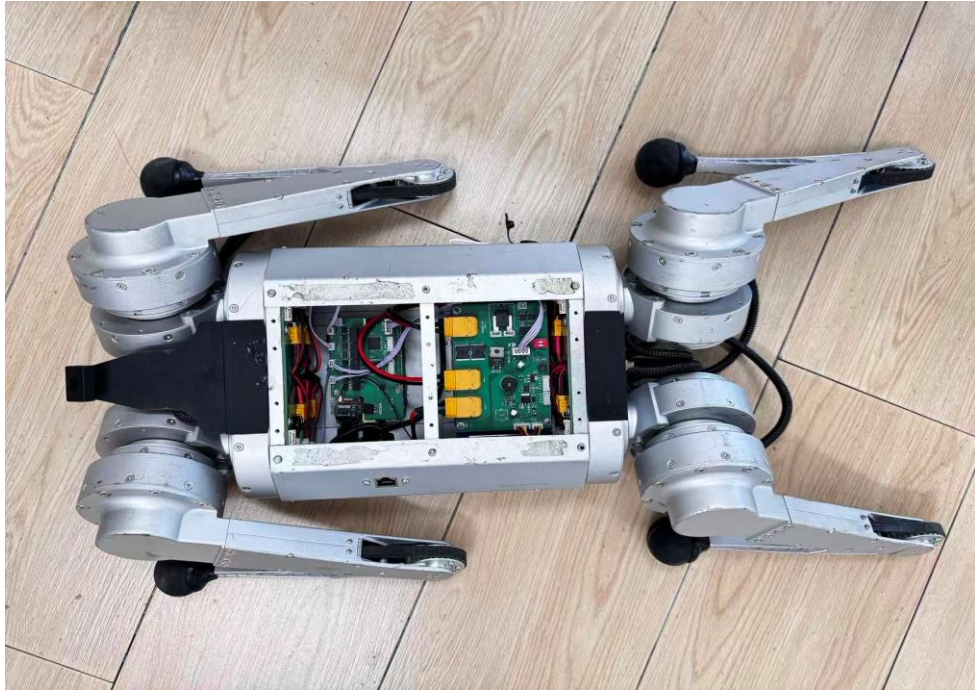


图2 内部电路图

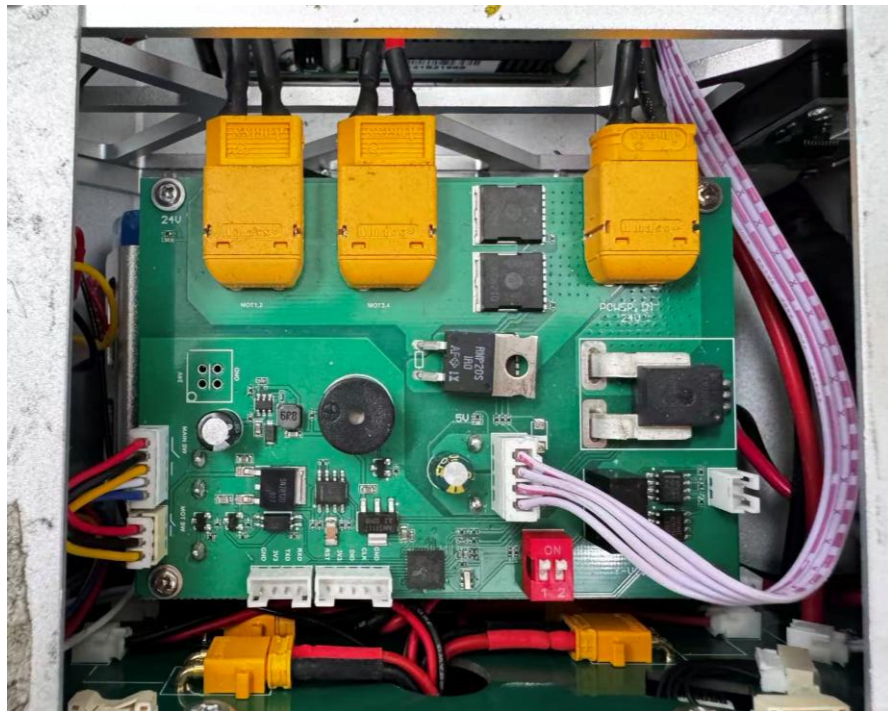


图3 内部电源板图

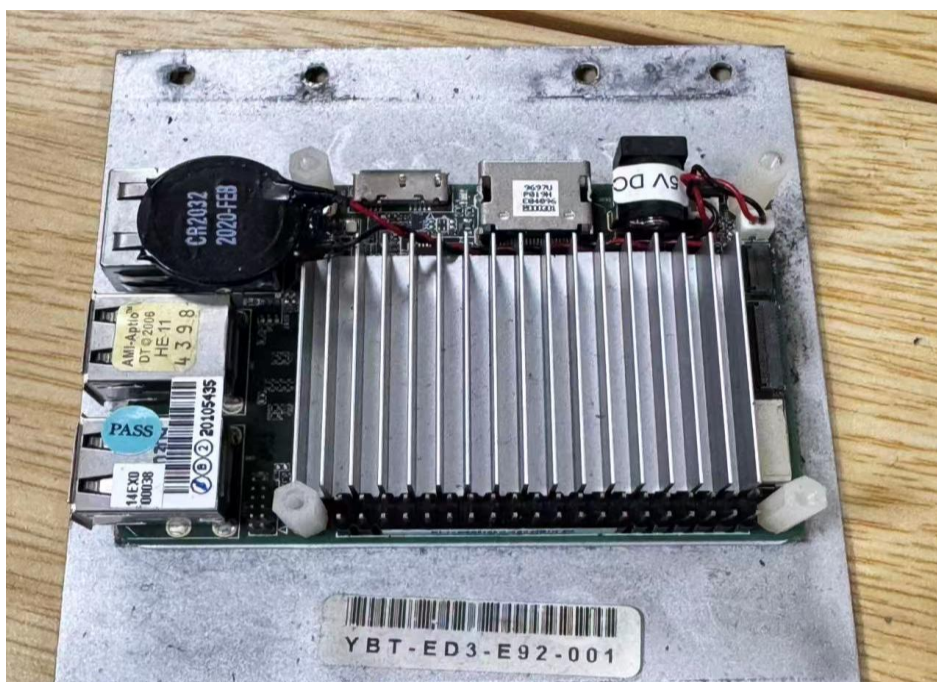


图4 机器狗上位视觉主控图



图5 机器狗下位视觉主控图（关节一体化伺服驱动轮毂）

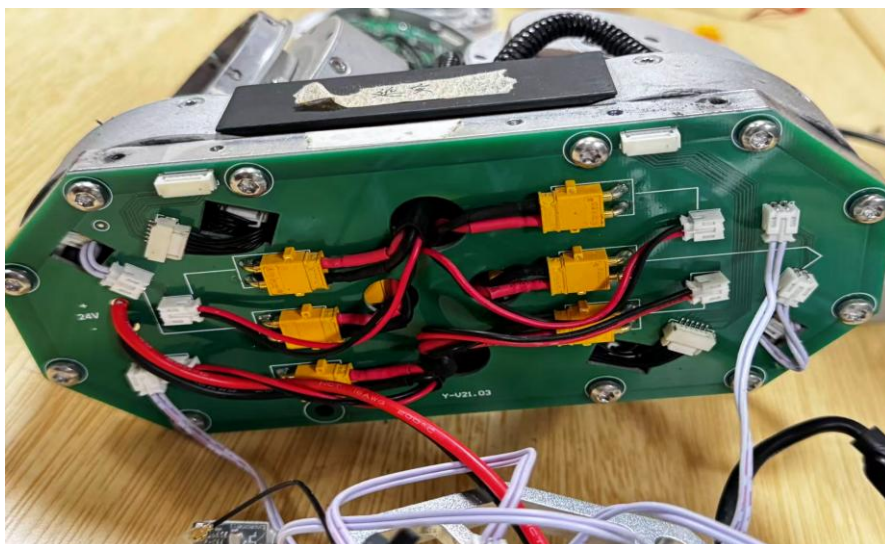


图6 机器狗电源分配、总线转接板图

十、附录

代码基于 Qt+OpenCV+LCM+UDP 开发，运行在 Linux(Ubuntu)平台，是四足机器人视觉循迹、自主运送任务的顶层入口程序，核心实现摄像头图像采集→视觉图像处理→运动指令下发→上位机交互全链路，适配快递运送赛事自主巡航需求。

主函数：

```
// =====  
// 主函数 - 程序入口  
// 功能概述：  
// 1. 解析命令行参数，设置运行模式和颜色标记  
// 2. 初始化摄像头  
// 3. 启动 LCM 通信线程和定时控制线程  
// 4. 进入主循环：采集图像 → 图像处理与运动控制 → LCM 指令发送 → 状态日志  
// 5. 可选：通过 UDP 与上位机通信，显示/设置颜色阈值  
// =====  
int main(int argc, char* argv[]) {  
    // 初始化 UDP 通信线程，用于接收上位机发来的颜色阈值设置等指令  
    UdpUtil udpsocket;  
    // ---- 命令行参数解析 ----  
    // 根据传入的参数选择运行模式以及是否将图像发送给上位机  
    // 参数 1 (inputParameters1)：指定运行模式  
    // 参数 2 (inputParameters2)：指定分岔颜色 / 显示图像标志  
    QString inputParameters1;  
    QString inputParameters2;  
    bool flag = false;  
    // 无参数：默认使用轨迹跟踪模式  
    if (argc == 1) {  
        printf("使用默认参数，模式为轨迹跟踪\n");  
        printf("模式：轨迹跟踪\n");  
        printf("默认驻留颜色为红色\n");  
        mythread.mode = track;  
        residenceColor = "brown";  
    }  
    // 有一个及以上参数  
    else if (argc >= 2) {  
        inputParameters1 = argv[1];  
        // 轨迹跟踪模式（无指定驻留颜色）  
        if (inputParameters1=="track") {  
            printf("模式：轨迹跟踪\n");  
            printf("默认驻留颜色为空\n");  
            mythread.mode = track;  
        }  
        // 轨迹跟踪模式 + 棕色驻留点  
        else if (inputParameters1=="brown") {  
            printf("模式：轨迹跟踪\n");  
            printf("输入的驻留颜色为棕色\n");  
            mythread.mode = track;  
            residenceColor = "brown";  
        }  
        // 轨迹跟踪模式 + 紫色驻留点
```

```

else if (inputParameters1=="violet") {
    printf("模式: 轨迹跟踪\n");
    printf("输入的驻留颜色为紫色\n");
    mythread.mode = track;
    residenceColor = "violet";
}
// 停止模式
else if (inputParameters1=="stop") {
    printf("模式: 停止\n");
    mythread.mode = stop;
}
// 无效参数, 默认使用轨迹跟踪模式
else {
    mythread.mode = track;
    printf("参数错误\n");
    printf("模式: 轨迹跟踪\n");
}
// 处理第二个参数
if (argc == 3) {
    inputParameters2 = argv[2];
    // 指定红色分岔路口
    if (inputParameters2=="red") {
        printf("模式: 轨迹跟踪\n");
        printf("输入的分岔颜色为红色\n");
        mythread.mode = track;
        divergerColor = "red";
    }
    // 指定绿色分岔路口
    else if (inputParameters2=="green") {
        printf("模式: 轨迹跟踪\n");
        printf("输入的分岔颜色为绿色\n");
        mythread.mode = track;
        divergerColor = "green";
    }
    // 启用图像显示功能, 发送图像到上位机
    else if (inputParameters2=="showImage") {
        printf("开始显示图像\n");
        flag = true;
        udpsocket.start();
    }
}
}
// ---- 初始化摄像头 ----
// 从索引 0 开始依次尝试打开摄像头, 最多尝试 4 次
for (int i = 0; i <= 3; i++) {
    cap.open(i);
    if (!cap.isOpened()) {
        printf("摄像头打开失败, 编号=%d\n", i);
        if (i == 3)
            return 0;
    }
}

```

```

    }
    else {
        printf("摄像头打开成功, 编号=%d\n", i);
        break;
    }
}
// ---- 初始化通信与控制线程 ----
// 创建 LCM 通信对象, 用于向机器人发送运动控制指令
lcmUtil *lcmutil = new lcmUtil;
// 启动定时控制线程, 管理各模式下的状态切换和延时
mythread.start();
// 用于测量循环耗时的计时器
QTime time;
// ---- 主循环 ----
// 持续采集图像、处理并发送运动指令
while (1) {
    // 摄像头断线重连检测
    // 当摄像头断开时, 循环尝试重新打开
    if (cap.get(CAP_PROP_HUE)==-1) {
        while (1) {
            QThread::msleep(100);
            for (int i = 0; i <= 3; i++) {
                cap.open(i);
                if (cap.isOpened()) {
                    break;
                }
            }
            if (cap.isOpened()) {
                break;
            }
        }
    }
    // 从摄像头读取一帧图像
    cap >> srcImage;
    // 图像处理与运动控制
    if (!srcImage.empty()) {
        // 执行视觉伺服: 根据当前模式和图像内容计算运动控制参数
        imageProcess(srcImage);
        // 通过 LCM 发送步态指令到机器人 (v_des、gait_type 等为全局变量, 在 imageProcess 中被
        // 修改)
        lcmutil->send(v_des, gait_type, step_height, stand_height, rpy_des);
        // 输出当前状态信息到控制台
        logMode();
        // ---- 上位机通信 (可选) ----
        // 当启用图像显示时, 将图像发送给上位机, 并处理上位机返回的颜色阈值指令
        if (flag) {
            // 发送图像到上位机 (destination=1), 参数依次为:
            // 参数 1: 输入图像
            // 参数 2: 输出目标 (0=本机显示, 1=发送到上位机)
            colorgroup.showPicture(srcImage, 1);

```



```

        colorgroup.start();
        // 处理上位机发来的指令
        if (udpsocket.ifReceiveInfoFlag != 0) {
            switch (udpsocket.ifReceiveInfoFlag) {
                case 1:
                    // 指令 1: 选择颜色并返回该颜色的当前阈值
                    std::cout << "选择颜色并返回该颜色阈值" << std::endl;
                    colorgroup.chooseColor(udpsocket.color);
                    colorgroup.sendColorThreadhold();
                    colorgroup.ifRunContinueFlag = true;
                    break;
                case 2:
                    // 指令 2: 设置当前选中颜色的阈值
                    std::cout << "设置颜色阈值" << std::endl;
                    colorgroup.setColorThreadhold(udpsocket.colorThreadhold);
                    break;
                case 3:
                    // 指令 3: 保存当前所有颜色阈值到文件
                    std::cout << "保存颜色阈值" << std::endl;
                    colorgroup.save();
                    break;
            }
            udpsocket.ifReceiveInfoFlag = 0;
        }
    }
    // 循环延时 20ms, 降低 CPU 占用率
    QThread::msleep(20);
}
printf("程序运行结束\n");
cap.release();
return 0;
}

```